fewer tokens, better action

关键词

- **Embodied Agent Harness**
- Python Runtime

> 🔶 **本文提出一个很有价值的问题**
>
> 如果 Python 的局部重试逻辑不足以处理复杂的、未知的环境变化,系统应该在什么时机将控制权交还给VLM? #📄/challenge
>
> - 当前框架允许程序条件判断,也允许VLM根据反馈重新规划,但这并不等于它已经解决了最优的**重新规划触发问题** #📄/gap
> - 这个问题涉及
>
> - 闭环控制
> - 分层决策
> - agent 自适应调用机制

abstract

- 背景: VLM代理通过视觉、动作primitives 控制机器人
- challenge: 模型重复调用产生无效数据导致大量 token 开销
  - 传统Tool Calling下机器人每做一步都要请示大模型
- 提出:**PyRUA-Lean**框架通过整合"反馈驱动的基元组合+选择性观测"来实现其交互式的代码执行能力
  - PyRUA-Lean 相当于大模型一次性编写一段带有条件判断和纠错逻辑的程序，让机器人自主执行多个步骤；
  - 只有需要重新决策时，才把必要的反馈返回给大模型。
  - 两个核心组件
    - Feedback-driven primitive composition: 减少 LLM 调用
    - Selective Observation:控制上下文开销
- 效果

指标	Tool Calling	PyRUA-Lean
任务成功率	63.1%	71.7%
平均 LLM 调用次数	17.0	8.7
平均输入 Tokens	788K	276K

核心组件: 反馈驱动的基元组合+返回 selective observation

> (fewer tokens, better action, p.1)
> We introduce PyRUA-Lean, an **interactive code-execution framework** that couples **feedback-driven primitive composition with selective observation**: the agent composes classical robot primitives and learned vision-language-action (VLA) policies into **Python cells** that perform conditional checks and **local retries**, returning only explicitly **requested images and state feedback** for **replanning**.
> 我们提出了 PyRUA-Lean，一种将反馈驱动的基元组合与选择性观测相结合的交互式代码执行框架。该框架使智能体能够将经典机器人操作基元与学习得到的视觉-语言-动作（VLA）策略组合为 Python 代码单元，在其中执行条件检查与局部重试，并且仅返回显式请求的图像和状态反馈，以支持后续的重新规划。

本文提出一种新的机器人控制算法, 对传统VLM agent 执行架构的改进

- 💎 大模型负责:高层规划
- Python 程序负责
  - 具体执行
  - 条件判断
  - 局部纠错
  - 避免把所有中间信息传回大模型

交互式代码执行框架

(fewer tokens, better action, p.1)
interactive code-execution framework

第 1 创新点:反馈驱动的基元组合:primitive composition 组合了什么?

(fewer tokens, better action, p.1)
feedback-driven primitive: 根据执行反馈，动态选择并组合基础操作单元，以完成复杂任务

primitive

in VLM: robot primitive

在机器人领域，Primitive 通常翻译为**基元**，也可以理解为基本操作单元

- action primitives: 动作基元,指代机器人控制中不可再分的基础动作单元或基本行为模块

  fewer tokens, better action, p.1

  > Vision language model (VLM) agents can control robots through visual feedback and action primitives

  桌上的苹果放进篮子可以分解为

  1. 定位苹果
  2. 移动机械臂到苹果附近
  3. 抓
  4. 移动到篮子上方
  5. 松爪

     每一步都可能由一个 primitive 实现

	Tool (RPent)	*Code* (PyRUA-Lean)
Primitives, the same in both		
Arm motion	`move_to`, `move_pose`, `move_delta`, `rotate_wrist`, `rotate_pitch`	
Gripper	`set_gripper`, `release`, `scripted_grasp`	
VLA policy	`pi0_pick`, `pi0_doubled`, `lingbot_act`, `rldx_skill`, `rldx_arm`	
Mobile base	`navigate_to`, `move_base`	
Perception	`segment`, `back_project`, `back_project_batch`, `sample_world_xyz`, `query_world_map`	
Observations		
State, success flag	with every move, or `view_env_state`	`robo.state()`, `robo.done`
Camera images	three with every move	when the code calls `robo.show`
World coordinates	through the perception tools	the same, or `robo.world_map()` as one array

Primitive	功能	类型
`move_to()`	移动机械臂到指定位置	经典控制基元
`move_delta()`	控制机械臂进行相对位移	经典控制基元
`set_gripper()`	控制夹爪	经典控制基元
`segment()`	进行目标分割和定位	感知基元
`back_project()`	将图像坐标反投影为三维坐标	感知基元
`pi0_pick()`	调用学习得到的 VLA 抓取策略	VLA Policy

- primitive 不一定是神经网络
  - 也可以是传统机器人控制
  - 也可以是深度学习模型

composition

❤**PyRUA-Lean**的创新并不在重新训练这些 primitive, 而在于如何组织和调用它们

- composition: 将多个 primitive组织成具有执行逻辑的完整程序

```
# 1. 定位苹果
apple = robo.segment("red apple")

# 2. 移动到苹果附近
result = robo.move_to(apple.world_xyz)

# 3. 根据执行结果决定下一步
if result.reached:        # 🔶注意这里的if

    # 调用 VLA 策略抓取
    pick = robo.pi0_pick("pick up the apple")

    # 检查抓取结果
    print(pick)
```

- `if`: 机器人并不是无条件执行预设的动作序列,而是能够根据前一步结果决定下一步
- 引出 **feedback-driven**

❤ feedback-driven: 反馈驱动意味着什么?

假设机器人执行 `result = robo.move_to(target)` 时, 机器人没有到达目标位置

传统做法

机器人执行➜ 返回结果➜ LLM 分析➜ LLM 再次调用工具

PyRUA-Lean

允许: 机器人执行➜Python 检查结果➜ Python 局部重试

```
for attempt in range(3):  # ②局部重试local retries

    result = robo.move_to(target)

    if result.reached:  # ①条件检查:动作是否成功
        print("Success")
        break

    print("Retrying...")  # ③feedback-driven composition 反馈驱动的组合
```

```
print(robo.task)
print(robo.success_criteria())
for attempt in range(3):
    result = robo.rldx_arm()
    print(result)
    print(robo.state())
    if robo.done or result.status != 'cap':
        break
if not robo.done:
    robo.show('agentview')
    robo.show('wrist')
```

♒传统

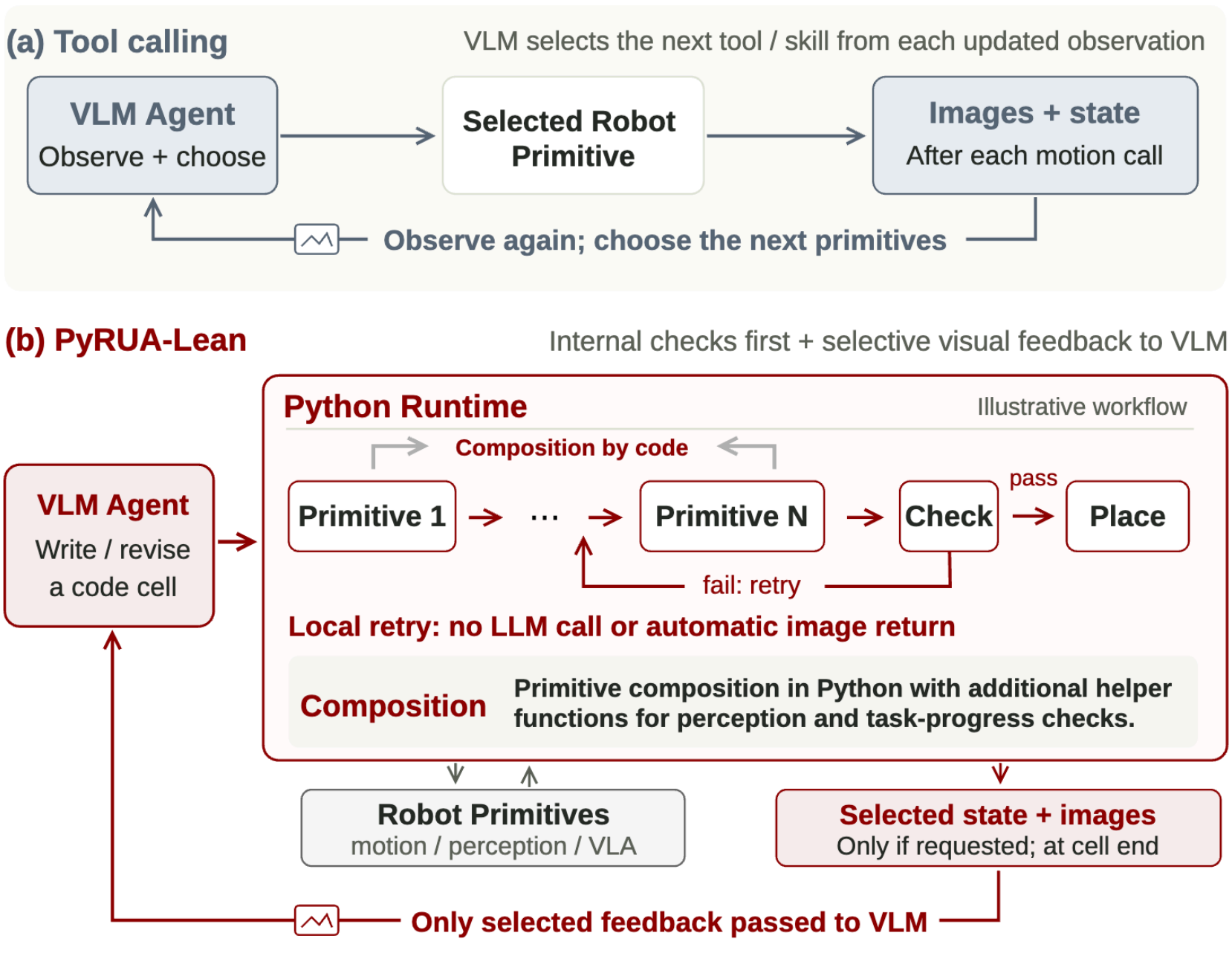
传统是一次性生成动作序列, 而反馈驱动可以根据前一步的执行反馈,决定继续、 重试、 停止

并不意味着每次失败都由 LLM 重新规划

作者希望将简单的执行反馈处理交给 python,从而减少 LLM 调用 (whose inference is time-consuming)

- 只有局部程序无法处理问题时,才需要返回更高层模型进行 replanning

> (fewer tokens, better action, p.3)
> These checks and branches execute within the cell without additional LLM invocations.
> `条件判断和分支可以在同一个python cell中执行,无需额外调用LLM`



Python cells: 为什么要强调 python 代码单元?

> (fewer tokens, better action, p.4)
> b) PyRUA-Lean instead generates Python cells that compose robot primitives with helper functions, conditional checks, and local retries.

涉及上图中传统Tool Calling 🆚 本文 code exexution

传统Tool Calling

> #### Tool Calling
>
> 传统机器人控制
>
> #### 🍎抓苹果放入篮子
>
> 传统 agent 执行流程

```
LLM
 ↓
调用 segment()
 ↓
返回苹果位置
 ↓
LLM
 ↓
调用 move_to()
 ↓
返回移动结果
 ↓
LLM
 ↓
调用 pi0_pick()
 ↓
返回抓取结果
 ↓
LLM
 ↓
调用 move_to()
```

> - pi0.pdf

每个依赖上一步结果的primitive的动作,通常需要新一轮LLM推理
从而带来两个问题:

1. LLM 调用次数多
   - 每次调用都有推理成本
2. 上下文不断增长
   - 机器人每次返回图片、状态和执行结果都有可能进入之后的模型上下文
   - 即使某些信息已经不重要,仍可能带来 token 开销

PyRUA-Lean 的 Code Execution

PyRUA-Lean 让 LLM 一次生成一段 Python 代码,例如

```
apple = robo.segment("apple")

if apple.found:

    move = robo.move_to(apple.world_xyz)

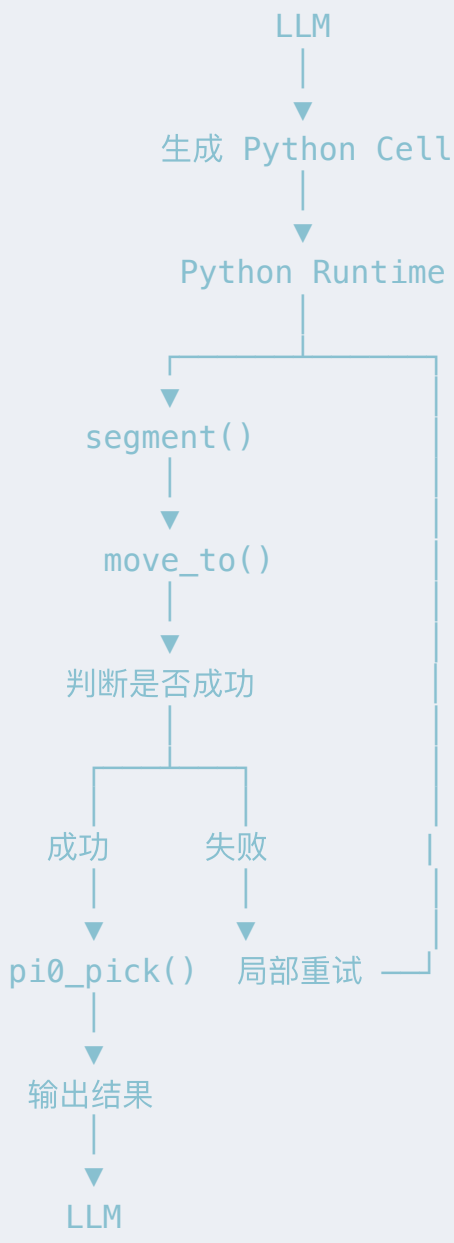
    if move.reached:

        pick = robo.pi0_pick("pick up apple")

        print(pick)
```

然后交给 python runtime 执行
整个代码单元内可以包含多个primitive调用

执行逻辑:



🔸 与传统Tool Calling不同:一次调用,可以完成多个相互依赖的机器人操作

作者的 python runtime 具有持久化的执行状态

> (fewer tokens, better action, p.3)
> At each interaction, the VLM generates a code cell, the runtime executes it, and the returned feedback informs the next code cell.

```
# Cell 1
apple = robo.segment("apple")
```

下一次调用:

```
# Cell 2
print(apple.world_xyz)
```

变量apple 仍然存在

- 这被称为 **持久化程序状态**
- #⌨️/gap 这种状态仅在当前任务 episode 保留,论文并未实现跨 episode 的经验记忆

第 2 核心机制:selective observation: 为什么选择性观测能够节省 token?

区分

- observation: 机器人通过传感器(摄像头、状态接口、感知模块)获得的信息
- **selective observation**: 只将明确需要的信息返回给VLM

传统框架问题

> (fewer tokens, better action, p.2)
> in systems such as RPent [33], motion primitives automatically return multiple camera images at the end of each step of execution

RPent的 Tool Calling 在每次运动操作后,会自动返回多个摄像头图像

- 机器人移动机械臂: `move_to(target)`
- 系统自动返回:

```
Image 1: 正面摄像头
Image 2: 侧面摄像头
Image 3: 腕部摄像头

Robot State: ...
```

- 然而,机器人可能只需要知道:
  - `reached = True`
  - 也就是说返回三张图像并不一定必要
  - 图像的输入反倒造成💎多模态模型的 token 开销 #⌨️/challenge
  - 增加推理成本

PyRUA-Lean 的选择性观测

作者运行程序明确制定哪些信息需要返回

```
result = robo.move_to(target)
if result.reached:
    print("Move succeeded") #当机器人成功移动时,🔶只返回文本信息
else:
    print("Move failed")
    robo.show("agentview") #当机器人移动失败时,程序才请求图像
```

- 机器人内部仍然可以调用segment()等感知模块处理图像
- selective observation 主要控制: 哪些执行结果和图像需要进入VLM的上下文
  - 降低 Tolem 消耗

> (fewer tokens, better action, p.4)
> The agent specifies which feedback to return through explicit output statements and camera requests such as robo.show.
> 只有显示输出的信息和请求的图像才会被返回给模型,而且这些樊哙在当前python结束时统一返回

replanning:为什么最后还要返回反馈?

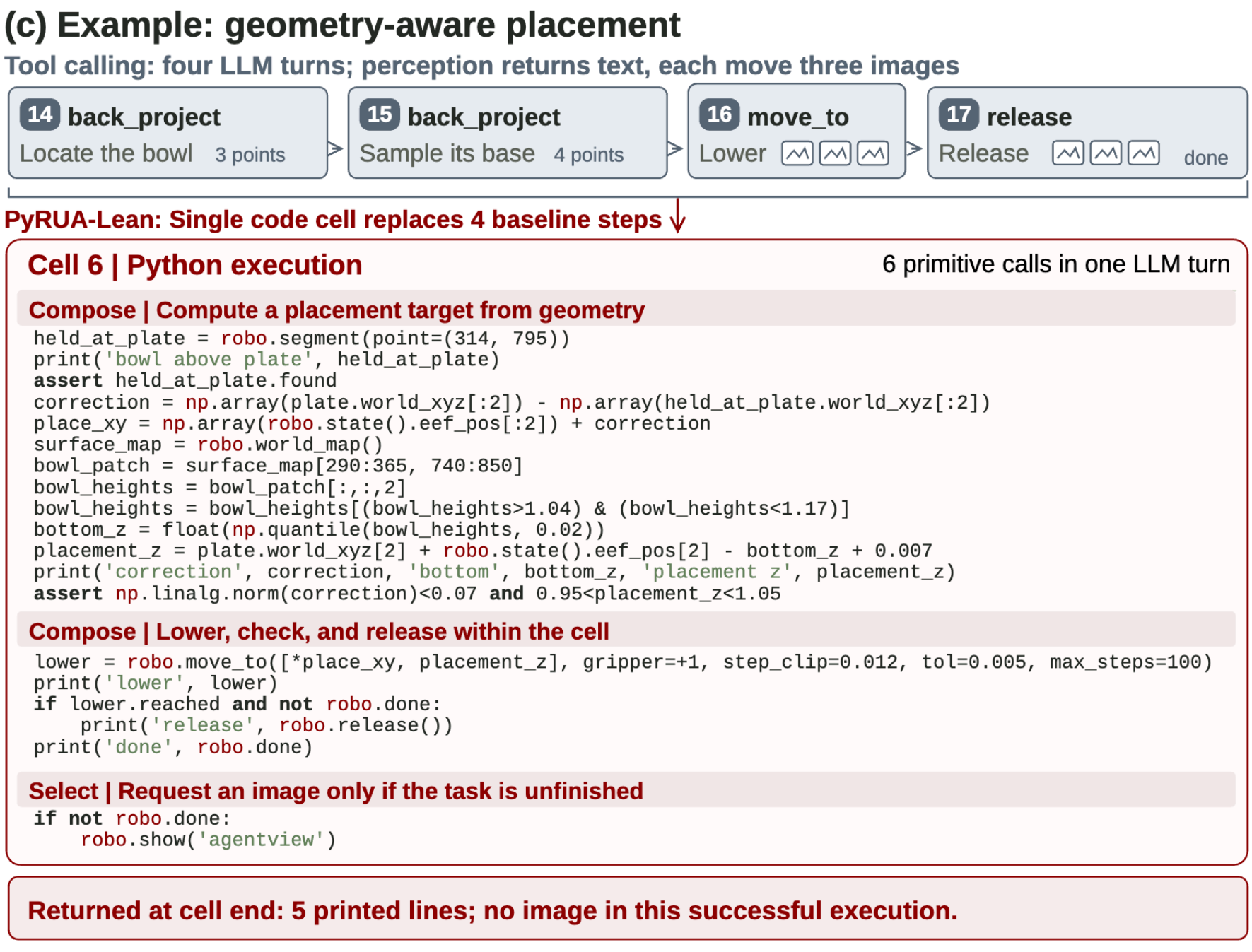
> (fewer tokens, better action, p.1)
> returning only explicitly requested images and state feedback for replanning.

既然 Python 已经能够条件判断和局部重试,为什么还需要 LLM?

- 原因是: **Python**只能执行已经编写好的处理逻辑,而 **LLM** 可以根据新情况制定新的计划
  - 机器人抓苹果,第一次失败 `if not result.reached: retry()`
    - 可由 Python 局部处理
  - 但如果出现:苹果从桌面滚落到地面
    - 原来的抓取策略可能不再适用
    - 此时需要: python runtime➜检测到异常➜请求摄像头图像➜ 返回VLM➜ 重新理解场景➜ 制定新的动作计划
      整个系统实际上具有两个层次的反馈处理机制

层次	执行主体	负责什么
局部执行层	Python Runtime	条件判断、局部重试、执行预设的恢复逻辑
高层规划层	VLM Agent (GPT-6 Astra)	理解新的场景、修改计划、生成新的 Python Cell

结合真实案例理解



目标: 把一个碗放在盘子上

- 传统Tool Calling需要四轮 LLM 交互
  - 计算目标位置
  - 进一步确定放置集合信息

    - 控制机械臂下降
    - 松开夹爪
- 而 PyRUA-Lean 将相关计算与动作放在一个 Python cell 中

```
# 计算放置位置
correction = np.array(plate.world_xyz[:2]) \
           - np.array(held_at_plate.world_xyz[:2])

# 根据几何信息计算下降高度
placement_z = ...

# 执行下降
lower = robo.move_to(...)

# 成功下降后才松开夹爪
if lower.reached and not robo.done:
    robo.release()

# 只有任务尚未完成时才请求图像
if not robo.done:
    robo.show("agentview")
```

体现两个创新点

1. primitive composition: 将几何计算、移动、 条件判断和释放操作组合起来
2. selective observation: 只有任务未完成时,才请求摄像头图像
   在这个成功案例中，程序没有返回图像，只返回了五行打印信息 `fig最后一行`

原本需要四轮 LLM 交互的过程，被压缩为一轮

科研创新点评价

论文创新的是 agent 执行接口

而不是VLA模型本身

- 本文重点不在于训练一个更强的 VLA
- 使用同样的 GPT-6 Astra 规划器、同样的底层机器人基元和冻结的 VLA 策略，主要改变了 LLM 与底层机器人能力的交互方式

从系统设计角度看

$$\boxed{\text{VLM Planning} \rightarrow \text{Python Execution} \rightarrow \text{Selective Feedback} \rightarrow \text{VLM Replanning}}$$

- 与传统 Tool Calling 相比，关键变化在于将大量中间控制流程从 LLM 上下文迁移至 Python Runtime

局限性

> (fewer tokens, better action, p.7)
> Fewer VLM invocations account for most of the token reduction
> `token的减少主要来源是LLM调用次数的减少,而不只是减少单词调用返回的图像`

- primitive composition 比 selective observation 更有效果
- 论文没有通过完全独立的消融实验证明两个机制各自的因果贡献 `#📃/gap`

🔶 本文提出一个很有价值的问题

如果 Python 的局部重试逻辑不足以处理复杂的、未知的环境变化,系统应该在什么时机将控制权交还给VLM? `#📃/challenge`

- 当前框架允许程序条件判断,也允许VLM根据反馈重新规划,但这并不等于它已经解决了最优的**重新规划触发问题** `#📃/gap`
- 这个问题涉及
    - 闭环控制

- 分层决策
- agent 自适应调用机制